Object Detection For Turkish Pharmaccutial Drug Dataset

Le Viet Hung, Hoang Kim Huong, Nguyen Gia Nam, Dang Dinh Hoa, Doan Ngoc Linh, Nguyen Duc Thanh
University of Science and Technology of Hanoi (USTH)
Lecturer: Tran Giang Son
Hanoi, March 2026

Introduction

Object detection plays a crucial role in many computer vision applications, especially pharmaceutical warehouse management systems, pharmacy automation, and drug inventory systems. In these scenarios, the ability to accurately detect and classify drug boxes reduces human error and improves operational efficiency.

However, detecting drug boxes also presents many challenges. Many types of drugs have similar shapes and sizes, packaging colors, and drug boxes often appear from different angles in images. This makes differentiating between drug classes more difficult than with conventional object detection problems. In this project, we try to solve these problems by using two object detection methods, RetinaNet and Fully Convolutional One-Stage Object Detector with ConvNext-Small Backbone.

Data overview

In this project, we use the Turkish Pharmaceutical Drug Dataset, a dataset built for object detection and compatible with the YOLOv8 model, including 12 object labels along with bounding box annotations.

The dataset includes 14.932 total images, collected from real-world photos of drug packagings and saved in **.jpg** format with a size of 640 × 640 pixels. The dataset includes 12 object classes, specifically:

1) *aferin*: Decongestant medication
2) *apranax*: Anti-inflammatory drug
3) *arveles*: Pain reliever
4) *aspirin*: Common pain reliever and blood thinner
5) *dolorex*: Pain medication
6) *hametanKrem*: Topical cream
7) *hametanMerhem*: Topical ointment
8) *majezik*: Pain reliever
9) *metpamid*: Anti-nausea medication
10) *parol*: Pain reliever
11) *terbisil*: Antifungal medication
12) *unisom*: Sleep aid medication

The dataset is divided into three main sets: training set, validation set, and test set, to serve the training, evaluation, and testing phases of the model. The dataset also contains some low-quality images, such as blurry, black and white, or grayscale images, as well as images with noise. Additionally, some images are horizontally flipped or randomly rotated. These characteristics should be considered and appropriately addressed during the data preprocessing steps to ensure the quality of the input data for the model.

Methodology and Implementation

I. Model: ConvNeXt + RetinaNet

A. Data Preprocessing

Data Cleaning and Annotation Standardization

In the multi-class medicine box detection task, the input data is quite diverse: images have different sizes, camera angles, lighting conditions, and label formats. Because of this, we preprocess the data before training to keep images and bounding boxes consistent. Images are loaded from .jpg, .jpeg, and .png files, then converted to RGB so that all inputs have 3 channels. After that, each image is converted to a tensor, and pixel values are scaled from [0,255] to [0,1]. This helps reduce value-scale differences between images and makes training more stable. The dataset has two label formats: YOLO bounding box (class x_center y_center width height) and polygon ($class, x_1, y_1, x_2, y_2, \ldots, x_n, y_n$). To make labels uniform, we convert all annotations to one format (bounding box). For polygon labels, we create a bounding box by taking the minimum and maximum values on the x and y axes. This gives a single,

consistent label format that matches the RetinaNet architecture.

Data Augmentation and Normalization

After label standardization, we run another cleaning step to remove incorrect annotations. If a box goes outside the image, we clip it to the image boundary. If a box is invalid (for example, too small or with no valid area), we remove it. We also check that class IDs are in the valid range. Next, we apply preprocessing and data augmentation. During training, all images are resized to 640×640, and random horizontal flip with probability 0.5 is used to increase data diversity and help the model learn spatial variations. Finally, images are normalized using ImageNet mean and standard deviation, which helps make the training process more stable.

Fig. 1: Data preprocessing pipeline

B. Methodology

The problem is formulated as a multi-class object detection task with 12 medicine box classes. For each input image, the model needs to predict a set of bounding boxes, the corresponding class labels, and their confidence scores. To solve this problem, we build an object detection system based on the RetinaNet architecture, combined with a ConvNeXt-Tiny backbone and a Feature Pyramid Network (FPN). The goal is to leverage the strong feature extraction capability of ConvNeXt together with the multi-scale detection mechanism of FPN in order to improve detection performance.

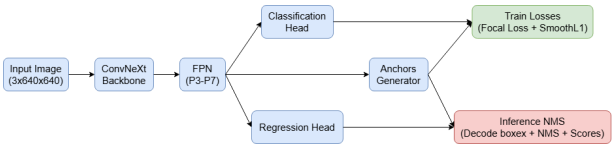


Fig. 2: Overall architecture of the RetinaNet–ConvNeXt object detection model

ConvNeXt

First, the input image is passed through the ConvNeXt backbone to extract spatial and semantic features at different levels. The shallow layers mainly capture low-level features such as edges and textures, while the deeper layers encode higher-level semantic information and object shapes. This hierarchical representation allows the model to learn discriminative features that help distinguish between medicine boxes that may look visually similar.

Feature Pyramid Network (FPN)

The features extracted from the backbone are then fed into the Feature Pyramid Network (FPN) to generate multi-scale feature representations. FPN uses a top-down pathway combined with lateral connections to merge information from different levels of the backbone. As a result, the model produces feature maps at multiple resolutions, ranging from (P_3, P_7). These multi-scale feature maps allow the detector to effectively identify small, medium, and large objects within the same image. On each feature map of the FPN, RetinaNet applies two parallel prediction branches: the classification head and the regression head. The classification head predicts the class probability of objects at each anchor location, while the regression head predicts the offsets of the bounding box relative to the anchor box. These heads operate on all levels of the feature pyramid to ensure effective multi-scale detection.

Anchor Generator

Next, an anchor generator is used to create anchor boxes at each location of the feature maps. At every position on the feature map, multiple anchor boxes with different scales and aspect ratios are generated to cover objects with various sizes and shapes. During training, these anchor boxes are compared with the ground truth bounding boxes using the Intersection over Union (IoU) metric in order to determine their labels. Based on the IoU values, anchors are assigned as follows:

- Positive anchor: IoU ≥ 0.5 with a ground truth box
- Negative anchor: IoU < 0.4
- Ignore: $0.4 \leq$ IoU < 0.5

The anchors labeled as positive are used to train the model to predict both the class probabilities (classification head) and the bounding box offsets (regression head).

Training Losses

During training, the model simultaneously optimizes two loss components: classification loss and regression loss. For the classification branch, Focal Loss is applied to address the class imbalance problem commonly encountered in dense object detection tasks. Focal Loss reduces the weight of easy samples and focuses more on difficult samples, thereby improving the effectiveness of model training. For the regression branch, Smooth L1 Loss is used to optimize the predicted offsets between the anchor boxes and the ground truth bounding boxes. This loss function reduces the impact of large errors and helps stabilize the training process.

Inference NMS

In the inference stage, the model first decodes box offsets from anchor coordinates to final bounding boxes. Then, each box is assigned a confidence score from the classification head. To remove duplicated predictions on the same object, we apply Non-Maximum Suppression (NMS): the highest-score box is kept, while overlapping boxes with IoU above a threshold are suppressed.

C. Implementation

The system is implemented in Python using PyTorch with a modular design, where each component of the pipeline is clearly separated to make training, evaluation, and future extension easier. The data module handles image loading, converts YOLO and polygon annotations into a unified bounding box format, and applies data augmentation depending on the phase (train, validation, or test). The model module includes the ConvNeXt backbone, the Feature Pyramid Network (FPN), the two prediction branches of RetinaNet (classification and regression heads), and a multi-scale anchor generation mechanism.

Training Configuration

The model is trained with several configuration settings to ensure stable and efficient training. Mixed precision training (AMP) is used to improve computational efficiency and reduce GPU memory usage. In addition, gradient clipping is applied to stabilize parameter updates during training.

Training Pipeline

Fig. 3: Training pipeline of the proposed RetinaNet–ConvNeXt detection model

In each training batch:

- The images and bounding boxes are fed into the backbone network to extract feature representations.
- The extracted feature maps are then passed to the Feature Pyramid Network (FPN) to generate a multi-scale feature pyramid.
- The RetinaNet heads predict the bounding boxes and the corresponding class probabilities.
- The loss is computed using Focal Loss for classification and Smooth L1 Loss for bounding box regression.
- Gradients are calculated through backpropagation, and the model parameters are updated accordingly.

Evaluation

After each epoch, the model is evaluated on the validation set using COCO metrics to compute evaluation scores such as Average Precision (AP) and Average Recall (AR). And the system monitors both training loss and validation loss over time in order to observe the convergence behavior of the model during training.

II. Model: FCOS + ConvNeXt

A. Data Preprocessing

A1. Label Format Detection

The raw dataset labels contain a mix of two annotation formats within the same dataset. Each label file may contain lines in either YOLO bounding box format or YOLO polygon segmentation format, and both formats can appear within the same split. A unified preprocessing pipeline was developed to handle both formats automatically on a per-line basis.

The preprocessing pipeline detects the format of each label line by counting the number of coordinate values that follow the class ID. A line with **exactly 4 values** is treated as a YOLO bounding box in the format (c_x, c_y, w, h) and is used directly. A line with **5 or more values** is treated as a polygon annotation, where the values represent alternating x and y vertex coordinates. If the polygon has an odd number of coordinate values, the trailing value is dropped before processing. Lines with fewer than 4 coordinate values are considered malformed and are skipped entirely.

A2. Polygon to Bounding Box Conversion

For polygon-format annotations, the axis-aligned bounding box is derived by computing the minimum and maximum values across all x and y coordinates of the polygon vertices. All coordinate values are first clipped to the range $[0, 1]$ to handle any out-of-bounds annotations. The bounding box corners are then computed as:

$$\begin{aligned} x_1 &= \min(x_s), \quad x_2 = \max(x_s), \\ y_1 &= \min(y_s), \quad y_2 = \max(y_s) \end{aligned} \tag{1}$$

The YOLO center format is then produced as:

$$\begin{aligned} c_x &= x_1 + \frac{w}{2}, \quad w = x_2 - x_1, \\ c_y &= y_1 + \frac{h}{2}, \quad h = y_2 - y_1 \end{aligned} \tag{2}$$

Any box where $w < 10^{-4}$ or $h < 10^{-4}$ is discarded as degenerate.

A3. Data Augmentation

During training, a rich set of augmentations is applied using the Albumentations library with bounding box-aware transformations. All images are first resized to 512 × 512 pixels. Horizontal flipping is applied with probability 0.5, vertical flipping with probability 0.2, and random 90° rotation with probability 0.3. Color augmentations include random brightness and contrast adjustment (probability 0.4), hue–saturation–value shifts (probability 0.3), Gaussian blur with a 3×3 kernel (probability 0.2), and CLAHE contrast enhancement (probability 0.2). All images are normalized using ImageNet statistics with mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225).

The bounding box parameters are set with a minimum visibility threshold of 0.3, meaning any box that is more than 70% occluded by an augmentation transformation is discarded. For validation and test sets, only resizing and normalization are applied to ensure fully reproducible evaluation metrics.

B. Methodology

B1. Backbone: ConvNeXt-Small

The feature extraction backbone is **ConvNeXt-Small**, a modern pure convolutional network that achieves performance comparable to Vision Transformers while maintaining the simplicity and computational efficiency of standard CNNs. ConvNeXt uses depthwise separable convolutions, inverted bottleneck blocks, and LayerNorm instead of BatchNorm throughout the network. The backbone is initialized with **ImageNet-1k pretrained weights** provided by torchvision.

The ConvNeXt-Small backbone produces four feature maps at different spatial resolutions. The stem and Stage 1 produce features at 1/4 of the input resolution with 96 channels (C_2). Stages 2, 3, and 4 progressively downsample by a factor of 2, producing feature maps C_3 (192ch, 1/8), C_4 (384ch, 1/16), and C_5 (768ch, 1/32). Features C_3, C_4, and C_5 are passed to the FPN neck.

B2. Neck: Feature Pyramid Network (FPN)

The Feature Pyramid Network takes the multi-scale features from the backbone (C_3, C_4, C_5) and produces five output feature maps P_3 through P_7, each with 256 channels. Lateral 1×1 convolutions reduce each backbone feature map to 256 channels, and top-down connections with nearest-neighbor upsampling fuse high-level semantic features from deeper layers with spatially precise features from shallower layers. A 3×3 convolution is applied to each merged feature map to produce the final FPN output.

Two additional levels are added beyond the backbone outputs. P_6 is produced by a stride-2 3×3 convolution applied directly to C_5, and P_7 is produced by a stride-2 3×3 convolution applied to P_6 with a ReLU activation. Each FPN level targets objects of a specific size range: P_3 targets objects smaller than 64 pixels, P_4 targets 64–128 pixels, P_5 targets 128–256 pixels, P_6 targets 256–512 pixels, and P_7 targets objects larger than 512 pixels.

B3. FCOS Detection Head

The FCOS detection head consists of two parallel 4-layer convolutional towers sharing weights across all FPN levels. Both the *classification tower* and the *regression tower* use 256 channels per layer with GroupNorm (32 groups) and ReLU activations. GroupNorm is preferred over BatchNorm in detection heads because it is independent of batch size.

Three prediction branches produce outputs at every pixel location in each FPN level:

leftmargin=1.5em

- **Classification branch**: outputs class probability logits of shape (B, 12, H, W), one logit per class per pixel.
- **Regression branch**: outputs four non-negative distance values (l, t, r, b) representing the distances from each pixel to the left, top, right, and bottom edges of the assigned ground truth box. A learnable per-level scale parameter is multiplied onto the raw regression output before exponentiation to ensure positive distances.
- **Centerness branch**: outputs a single scalar per pixel representing how centered the pixel is within its assigned ground truth box.

During inference, the final detection score is computed as:

$$s_{\text{final}} = \sigma(\text{cls}) \times \sigma(\text{ctr}) \tag{3}$$

where $\sigma(\cdot)$ denotes the sigmoid function. This joint score suppresses low-quality predictions near object boundaries without requiring any additional post-processing.

B4. Target Assignment

During training, each pixel (x, y) in FPN level l is assigned to ground truth box b if and only if:

leftmargin=2em

1) The pixel falls strictly inside the box: $l > 0,\ t > 0,\ r > 0,\ b > 0$.
2) The maximum regression target $\max(l, t, r, b)$ lies within the scale range $[s_l^{\min},\ s_l^{\max})$ of level l.

If a pixel satisfies both criteria for multiple boxes, it is assigned to the smallest box by area. The centerness target is computed as:

$$\mathrm{ctr}^* = \sqrt{\frac{\min(l^*, r^*)}{\max(l^*, r^*)}} \times \sqrt{\frac{\min(t^*, b^*)}{\max(t^*, b^*)}} \tag{4}$$

B5. Loss Functions

Focal Loss (Classification)

Focal Loss addresses the severe foreground/background class imbalance in dense per-pixel detectors by down-weighting easy, well-classified background examples:

$$\mathcal{L}_{\mathrm{focal}}(p_t) = -\alpha_t \, (1 - p_t)^{\gamma} \times \log(p_t) \tag{5}$$

where $\alpha = 0.25$ and $\gamma = 2.0$. The modulating factor $(1 - p_t)^{\gamma}$ approaches zero for easy examples and equals 1 for completely misclassified examples. The total focal loss is normalized by the number of positive pixel locations in the batch.

GIoU Loss (Regression)

Generalized Intersection over Union (GIoU) loss directly optimizes box overlap rather than coordinate distances:

$$\mathcal{L}_{\mathrm{GIoU}} = 1 - \mathrm{IoU} + \frac{|C \setminus (A \cup B)|}{|C|} \tag{6}$$

where C is the smallest enclosing box of both the predicted box A and the ground truth box B. The penalty term ensures meaningful gradient flow even when predicted and ground truth boxes have zero overlap. Regression loss is computed only at positive pixel locations.

Centerness Loss

Binary Cross-Entropy loss is applied between the predicted centerness logits and the geometric centerness targets:

$$\mathcal{L}_{\mathrm{ctr}} = -\big[\mathrm{ctr}^* \log \hat{c} + (1 - \mathrm{ctr}^*) \log(1 - \hat{c})\big] \tag{7}$$

The total training loss is the unweighted sum of all three components:

$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{focal}} + \mathcal{L}_{\mathrm{GIoU}} + \mathcal{L}_{\mathrm{ctr}} \tag{8}$$

C. Implementation

C1. Optimizer and Learning Rate

The model is trained using the **AdamW optimizer**, which combines the adaptive learning rate properties of Adam with decoupled weight decay regularization. A weight decay of 10^{-4} is applied. The FPN and detection head are trained with a learning rate of 2×10^{-4}, while the backbone uses a lower learning rate of 2×10^{-5} after it is unfrozen. Gradient clipping with a maximum norm of 1.0 is applied at every update step. The learning rate follows a **CosineAnnealingLR schedule** that smoothly decays from the initial value to a minimum of 10^{-6} over 30 epochs.

C2. Two-Phase Training Strategy

Training is divided into two phases. During the **warm-up phase** (epochs 1–2), the ConvNeXt backbone is frozen and only the FPN and detection head parameters are updated. This prevents the randomly initialized head from producing large gradients that could corrupt the pretrained backbone features before meaningful detection patterns have been learned.

After epoch 2, the backbone is **unfrozen** and added to the optimizer as a separate parameter group with the reduced backbone learning rate of 2×10^{-5}. From this point, the entire network is trained end-to-end, with the lower backbone learning rate ensuring the pretrained features are adapted gradually rather than overwritten abruptly.

C3. Mixed Precision Training

PyTorch Automatic Mixed Precision (AMP) is enabled throughout training using `torch.amp.autocast` and `torch.amp.GradScaler`. Forward passes, loss computations, and activations use 16-bit floating point (float16), while gradient accumulation and parameter updates use 32-bit floating point (float32). This provides approximately **2× speedup** on the Tesla T4 GPU and reduces GPU memory consumption by roughly 40%, enabling a batch size of 8.

C4. Early Stopping

Early stopping monitors the validation loss after each epoch. If the validation loss does not improve by at least $\delta = 10^{-4}$ compared to the best observed value, a patience counter increments. Training stops automatically when the patience counter reaches **7 epochs** without improvement, and the best model weights are saved to disk at each new best validation loss.

C5. Hyperparameter Summary

All key hyperparameters used in the final training run are summarized below:

Parameter	Value	Description
Input image size	512×512 px	Resized from original resolution
Batch size	8	Images per gradient update
Optimizer	AdamW	Adaptive LR + weight decay
Head learning rate	2×10^{-4}	FPN + detection head
Backbone LR	2×10^{-5}	After unfreeze (epoch 3+)
Weight decay	10^{-4}	L2 regularization
Gradient clip norm	1.0	Prevents exploding gradients
LR scheduler	CosineAnnealingLR	$\eta_{\min} = 10^{-6}$
Freeze epochs	2	Backbone warm-up
Total epochs	30	Early stopping if triggered
Early stop patience	7	Epochs without improvement
Early stop min delta	10^{-4}	Minimum improvement threshold
Mixed precision	ON (float16)	AMP forward, float32 gradients
Focal α	0.25	Class balance weight
Focal γ	2.0	Hard example focus factor
Score threshold	0.05 (eval)	Minimum confidence
NMS IoU threshold	0.50	Suppression threshold
Max detections	100	Per image

Evaluation and Comparison

III. Result and Evaluation

A. Quantitative Evaluation

A1. mAP and AR Comparison

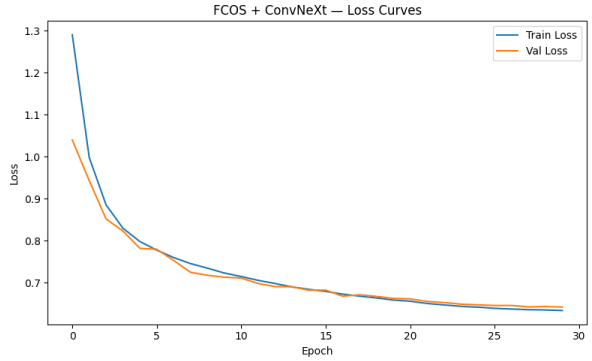
Table I summarizes the final evaluation metrics for both models.

TABLE I: Comparison of detection metrics on the test set (30 epochs).

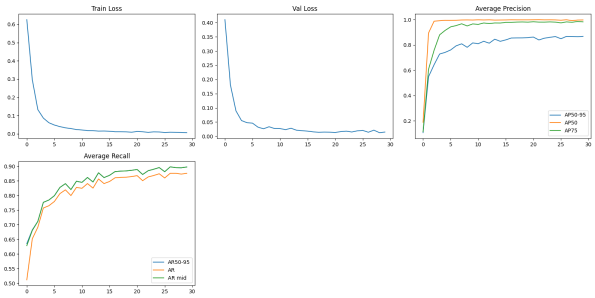
Metric	RetinaNet + ConvNeXt-Tiny	FCOS + ConvNeXt-Small	Δ
Train Loss (final)	≈ 0.01	≈ 0.634	—
Val Loss (final)	≈ 0.015	≈ 0.642	—
mAP@0.50:0.95	≈ 0.87	0.4740	+0.396
mAP@0.50	≈ 0.99	0.8224	+0.168
mAP@0.75	≈ 0.98	0.4948	+0.485
AR@0.50:0.95	≈ 0.90	0.5561	+0.344

RetinaNet + ConvNeXt-Tiny achieves substantially higher performance across all metrics. The most striking gap is at the strict **mAP@0.75** threshold, where RetinaNet reaches ≈ 0.98 compared to FCOS at 0.4948 — a gap of +0.485. This indicates that RetinaNet produces significantly tighter bounding box predictions that overlap more precisely with ground truth annotations. At the standard **mAP@0.50** threshold, RetinaNet reaches near-perfect ≈ 0.99 versus FCOS at 0.8224. The COCO-standard **mAP@0.5:0.95** shows an even larger gap of +0.396, confirming that RetinaNet's advantage is not limited to a single IoU threshold.

A2. Training and Validation Loss



(a) FCOS + ConvNeXt-Small loss curve.



(b) RetinaNet + ConvNeXt-Tiny training metrics.

Fig. 4: Training convergence comparison. Left: FCOS loss curves Right: RetinaNet loss curves and mAP/AR metrics

The loss curves reveal a fundamental difference in convergence behavior between the two models. The FCOS model starts with a relatively high training loss of approximately 1.29 and decreases steadily, reaching a final training loss of ≈ 0.634 at epoch 30. The validation loss mirrors the training loss closely (final ≈ 0.642), indicating no overfitting. However, the absolute loss values remain high because the FCOS loss is a composite of three components (Focal Loss, GIoU Loss, and centerness BCE) that sum to non-trivial values even for well-performing models.
In contrast, the RetinaNet model converges to a training loss of ≈ 0.01 and a validation loss of ≈ 0.015, both approximately two orders of magnitude lower than FCOS. This is partly because Smooth L1 Loss converges to near-zero for well-fitted boxes, whereas GIoU Loss retains a baseline value of $(1 - \text{IoU})$ which does not approach zero unless the predicted and ground truth boxes are identical. The RetinaNet metrics plot also shows that mAP@0.50 and mAP@0.75 both plateau near 1.0 after approximately 10 epochs, confirming rapid and stable convergence.

B. Discussion: Why RetinaNet Outperforms FCOS

The large performance gap between RetinaNet and FCOS on this dataset can be attributed to several interconnected factors spanning architecture, training configuration, and task characteristics.

B1. Higher Input Resolution

RetinaNet uses an input resolution of 640×640 compared to 512×512 for FCOS. For pharmaceutical drug detection, where the discriminative features often include fine-grained text, logos, and color patterns on drug packaging, higher resolution preserves more spatial detail throughout the feature pyramid. This directly benefits classification accuracy and bounding box localization, particularly for smaller objects or boxes that occupy a small fraction of the image.

B2. Anchor-Based Regression Stability

RetinaNet's anchor-based regression formulates the prediction task as estimating *offsets* relative to well-initialized anchor boxes. Since anchors are already close to the target box size and position, the model only needs to learn small residual corrections. This makes the regression problem easier to optimize, especially in early training epochs. FCOS, by contrast, predicts absolute distances (l, t, r, b) from each pixel to the box edges, which requires the model to learn the full mapping from pixel location to box coordinates. Although FCOS includes a per-level learnable scale parameter to help calibrate these distances, the regression problem is inherently harder, which explains the much larger gap between mAP@0.50 and mAP@0.75 observed for FCOS.

B3. Smooth L1 vs. GIoU Loss for Regression

RetinaNet uses **Smooth L1 Loss** for box regression, which penalizes coordinate errors independently along each axis. For well-separated classes with consistent box sizes (as in this pharmaceutical dataset), Smooth L1 converges quickly to near-zero values once the offsets are small. FCOS uses **GIoU Loss**, which has a theoretical minimum of $(1 - \text{IoU}) \geq 0$ even for correct predictions, meaning it cannot reach zero unless the predicted and ground truth boxes are exactly identical. Although GIoU is a better metric for direct overlap optimization and handles non-overlapping boxes more gracefully, it is harder to optimize to near-perfect values on fine-grained localization tasks, which contributes to the lower mAP@0.75 of FCOS.

B4. Backbone Capacity vs. Input Resolution Trade-off

FCOS uses ConvNeXt-Small (49.6M parameters) while RetinaNet uses ConvNeXt-Tiny (28.6M parameters). Despite having a smaller backbone, RetinaNet achieves better results, indicating that input resolution and regression formulation have a stronger impact than raw backbone capacity for this specific dataset. The higher resolution of 640×640 effectively compensates for the smaller backbone by providing richer spatial information to each FPN level.

B5. Missed Detections vs Confidence Threshold

The FCOS confusion matrix shows a high rate of missed detections, particularly for Lantus (56%), Flagyl (45%), and Augmentin (32%). These misses occur because the centerness-weighted confidence score in FCOS is more conservative: predictions near object edges have their confidence suppressed by the centerness factor, which can cause valid detections to fall below the confidence threshold of 0.3 used during evaluation. RetinaNet's anchor-based confidence scores are not subject to this suppression, leading to higher recall across all classes.

B6. Summary

Factor	RetinaNet Advantage	FCOS Limitation
Input resolution	640^2 (higher detail)	512^2 (lower detail)
Regression method	Offset from anchors (easy)	Abs. distance (harder)
Reg. loss	Smooth L1 (fast convergence)	GIoU (harder to zero)
Confidence scoring	Direct cls score	cls $\times$ centerness
Backbone	ConvNeXt-Tiny (efficient)	ConvNeXt-Small (larger)
mAP@0.75 result	≈ 0.98	0.4948

Conclusion

In this work, we trained and evaluated two object detection models on the Turkish Pharmaceutical Drug Dataset: RetinaNet + ConvNeXt-Tiny and FCOS + ConvNeXt-Small. RetinaNet performed better, achieving mAP@0.50 of ≈ 0.99 and mAP@0.75 of ≈ 0.98, while FCOS reached 0.82 and 0.49 respectively. The main reasons are the higher input resolution used by RetinaNet (640×640 vs. 512×512) and its anchor-based regression, which is simpler to optimize when object sizes are relatively consistent. FCOS remains a strong model, but its centerness scoring and GIoU loss made it harder to reach the same precision on this task. Both models converged well without overfitting. For future work, closing the gap is achievable by increasing FCOS input resolution and experimenting with better regression losses.